

听见·鼓浪屿

Gulangyu.AI 无障碍数字孪生导航系统

完整开发执行手册 v3.0

代码优化 → 用户测试 → 仿真验证 → 数据分析 → 作品集包装

当前项目状态总览

后端错误处理

全局异常捕获 + /api/health 接口

已完成

用户会话日志系统

user_sessions + query_logs 双表记录

已完成

NLP意图识别容错 5级模糊匹配 + Levenshtein编辑距离	已完成
多策略路径规划 综合最优 / 距离最短 / 最平坦 三条路	已完成
用户类型差异化权重 normal/elderly/disabled/blind 四档系数	已完成
真实用户小样本测试 5人测试 + 缺陷发现	下一步
1000条模拟压测 Python脚本自动生成测试用例	待执行
Unity A/B仿真测试 对照组 vs 实验组	待执行
数据分析与报告 图表生成 + 技术报告写作	待执行

① 启动后端并验证健康状态

打开终端，依次执行以下命令，确认所有接口正常响应。

终端命令

```
cd E:\my-gulangyu\backend
python main.py
# 另开一个终端，验证健康接口：
curl http://127.0.0.1:8000/api/health
# 期望返回（全部为true才算通过）：
# { "status": "ok", "db_connected": true, "node_count": 10, "graph_loaded": true }
```

② 验证核心接口

必须通过的3个测试

- 访问 <http://127.0.0.1:8000/docs> → 能看到Swagger UI页面
- POST /api/route 输入两个已知节点ID → 返回success:true和steps列表
- POST /api/route 输入不存在的地名 → 返回友好错误提示（不是500报错）

③ 打开日志监控

真实用户测试期间需要实时看到日志输出，提前开好终端窗口。

查看实时日志

```
# 测试结束后运行，生成报告：
python backend/test_report.py
# 输出：test_report.csv 和 accuracy_chart.png
```

为什么必须先做真实用户测试？

- 自己生成的1000条模拟数据会下意识避开系统不能处理的情况，数据失真
- 哪怕只有5个真实用户说的50条输入，价值远超1000条自造数据
- 先发现真实缺陷 → 修复 → 再跑模拟，模拟数据才有意义

① 招募5个测试用户（今天就能做）

用户类型	人数	招募渠道	特别说明
普通健康成年人	2人	同学/朋友	扮演普通游客，随意输入
老年人（55岁以上）	1人	父母/亲戚	观察他们如何表达目的地
非本地人（普通话）	1人	外省同学	测试方言/口音容错
任意用户（闭眼）	1人	任意	闭眼用语音输入，测试盲人模式

② 测试流程（每人约20分钟）

☐	开启会话记录 POST /api/session/start，记录session_id	P0
☐	让用户说出5个目的地 不要提示正确写法，观察自然输入（含错别字/简称/口音）	P0
☐	记录每次识别结果 系统识别对了吗？测试员在旁边标注is_correct	P0
☐	测试多策略路径选择 问用户：三条路线哪条更容易理解？反馈记录在notes字段	P1
☐	测试语音播报 播放AI生成的导航指令，用户能听懂方向吗？	P1
☐	关闭会话 POST /api/session/end，记录end_time	P0

③ 测试结束后立即运行报告脚本

生成测试报告

```
python backend/test_report.py
# 重点关注输出中的失败案例清单：
# fail_cases.csv 里每一条都是需要修复的真实缺陷
# 验收标准（未达标需修复后重测）：
# 意图识别准确率 >= 85%
# 平均响应时间 <= 500ms
```


修复优先级判断规则

- P0（必须修）：超过2个用户都遇到的问题，或导致系统崩溃的Bug
- P1（应该修）：只有1个用户遇到，但影响体验的问题
- P2（记录即可）：边缘情况，放入known issues说明，不影响测试进行

常见预期缺陷及修复方向

预期缺陷	修复方向	Trae Prompt关键词
用户输入简称系统不识别（如"协和"找不到"协和礼拜堂"）	在place_aliases表补充更多简称映射	"在place_aliases表新增以下别名..."
编辑距离阈值太严（差2个字就识别失败）	把Levenshtein阈值从2放宽到3，同时降低置信度	"将recognize_place函数的距离阈值改为3"
路径步骤描述太技术化（用户听不懂"节点ID"）	优化Prompt，改为口语化导航指令	"修改/api/chat的System Prompt使输出更口语化"
响应时间超过500ms	检查是否每次都重建图应使用全局单例	"确认route_engine是否为全局单例初始化"

① 理解测试目标

这个阶段在验证

- 验证NLP意图识别的鲁棒性
- 生成混淆矩阵和准确率图表
- 证明系统比直接查数据库容错率更高
- 产出可放进作品集的数据图

这个阶段不验证

- 不是真实用户数据（是模拟数据）
- 需要在真实用户测试后才跑（用真实缺陷指导测试用例设计）
- 不测试路径规划（那是Unity的工作）
- 数量1000是经验值，多多益善

② 在Trae中生成测试脚本

给Trae的Prompt（直接复制使用）

在 E:\my-gulangyu\backend\ 新建 stress_test.py 脚本。

功能：自动生成1000个测试用例并运行，统计意图识别准确率。

测试用例生成规则（从数据库读取所有建筑名称，对每个名称生成以下变体）：

1. 正确写法（基准）
2. 随机删除1个字（如'协和礼拜堂'→'协礼拜堂'）
3. 随机替换1个同音字（维护一个常见同音字映射表）
4. 取前两个字作为简称（如'协和礼拜堂'→'协和'）
5. 加入无关前缀（如'我想去协和礼拜堂'）

每个测试用例调用 nlp_engine.recognize_place() 函数。

统计输出：总准确率、各匹配级别命中率、失败案例Top10。

生成两个文件：stress_test_results.csv 和 accuracy_chart.png

accuracy_chart.png 包含：条形图（各级别命中率）+ 混淆矩阵热图。

③ 运行压测并记录结果

执行命令

```
cd E:\my-gulangyu\backend
```

```
python stress_test.py
```

```
# 运行时间约2-5分钟
```

```
# 完成后查看：
```

```
# - accuracy_chart.png（作品集图表）
```

```
# - stress_test_results.csv（详细数据）
```

```
# 验收标准：
```

```
# 总体准确率 >= 85%（低于则返回Phase 2修复）
```

```
# 级别1+2命中率 >= 60%（精确匹配+别名匹配占大头）
```

```
# 平均响应时间 <= 100ms（纯本地计算应该很快）
```

关键设计修正（与旧版SOP的区别）

- 旧版错误：A组用NavMesh随机寻路，会被NavMesh本身阻挡，无法真实对比
- 新版正确：A组和B组都用Dijkstra寻路，区别只是权重不同
- A组权重：只用distance（忽略坡度和摩擦力）→ 追求最短路，不管危险
- B组权重：你的完整复合权重（距离+坡度²+摩擦力+用户类型系数）
- 这样对比才能真实体现你的算法在特殊群体场景下的优势

① 准备Unity场景（3天）

☐	完成NavMesh烘焙 定义可通行区域，坡度陡的地方设为Non-Walkable边界	P0
☐	配置物理材质 青石板/水泥/红砖三种Physic Material，设置不同摩擦系数	P0
☐	创建A组Agent预制体 挂载NavMeshAgent，权重函数只用distance	P0
☐	创建B组Agent预制体 挂载NavMeshAgent，权重函数用完整复合权重（调用后端API）	P0
☐	挂载空间音频 7个核心节点挂载AudioSource，放入实地录制音效	P1
☐	添加危险节点碰撞记录 每个Agent触碰高危区域时调用 POST /api/record_hazard	P0

② 编写AutoAgent压测脚本（1天）

给Trae的Prompt（C#部分）

在 E:\Gulangyu_3D\Assets\Scripts\ 新建 ABTestController.cs。

功能：自动化A/B测试控制器。

实现要求：

- 从Inspector面板接受：A组数量(50)、B组数量(50)、测试节点列表
- 游戏开始时同时生成A组和B组Agent
- 每个Agent随机选择起点和终点（从测试节点列表中选）
- A组Agent：调用本地Dijkstra，权重=distance only
B组Agent：调用 GET http://127.0.0.1:8000/api/route?user_type=blind
- 记录每个Agent的：碰撞次数、路径长度、完成时间、是否触发危险节点
- 所有Agent完成后自动调用 POST /api/ab_test_summary 上报汇总数据
- 在Unity Console打印对比结果

③ 运行测试并录制视频（2天）

☐	先跑10个Agent测试 5A+5B，确认数据正常写入数据库再加量	P0
☐	正式跑100个Agent 50A+50B，同时观察后端终端有无报错	P0
☐	录制分屏对比视频 左侧Unity场景，右侧Web端热力图实时更新	P0
☐	运行数据分析脚本 python backend/data_analysis.py 生成对比图表	P0
☐	截取关键帧截图 A组Agent聚集在危险节点 vs B组Agent绕行的对比截图	P1

④ 验收标准

A/B测试必须达到的数据门槛

- B组危险节点触发率 < A组触发率的70%（B组更安全）
- B组路径长度 >= A组路径长度（B组绕开危险，路会更长，这是正常的）
- 后端在100个并发Agent时无崩溃（所有数据成功写入数据库）
- Web端热力图能正确显示高危区域（和实地测绘的危险节点位置吻合）

① 整合三类数据

数据来源	核心指标	用于证明什么	放入哪里
真实用户测试 (Phase 1)	意图识别准确率 平均响应时间 失败案例分类	系统在真实场景 的可用性	PS文书 用户研究部分
1000条模拟压测 (Phase 3)	混淆矩阵 各级别命中率 极端情况处理	NLP算法的 鲁棒性	作品集技术图表 简历技能描述
Unity A/B测试 (Phase 4)	B组危险触发率↓ 热力图分布 并发处理能力	空间算法的 实际效果	Demo视频 系统架构图

② 技术报告结构（作品集用）

☐ 第一节：问题陈述	为什么视障人士在鼓浪屿的导航是一个未被解决的问题	P0
☐ 第二节：系统架构图	Unity → FastAPI → SQLite → React 完整数据链路图	P0
☐ 第三节：算法设计决策	为什么选Dijkstra而非A*，复合权重公式的推导	P0
☐ 第四节：实验结果	三类数据的图表汇总，含验收标准对比	P0
☐ 第五节：局限性与未来方向	室内GPS失效问题，IMU-PDR方案展望	P1

③ Demo视频脚本（2分钟版本）

时间	画面内容	旁白要点
0:00-0:20	鼓浪屿实景 + 问题描述字幕	"世界文化遗产对视障人士意味着什么？"
0:20-0:50	Web端用户界面演示 语音输入"协和礼拜堂"	"多级容错识别，即使说错也能听懂你"
0:50-1:20	Unity场景 A/B分屏对比 A组乱撞 vs B组平稳绕行	"加权算法为特殊群体规划最安全路径"
1:20-1:50	Web管理端热力图 实时更新危险节点分布	"数字孪生让城市设计者看到真实风险"
1:50-2:00	系统架构图 + 项目信息	"先测后建，用数据改善无障碍环境"

① PS文书核心叙事框架

PS文书开篇建议

- 用一个具体的感知失败时刻开头：
 - "当我第一次踏上鼓浪屿的石板路，我意识到这座被列入世界遗产的岛屿，
 - 对于一位视障游客而言，不是历史，而是障碍。"
 -
- 三段论结构：
 - 1. Problem：用建筑师的空间语言描述问题（你独有的视角）
 - 2. Solution：用工程师的技术语言描述算法决策（重点说为什么这样选）
 - 3. Gap：诚实说明局限性，以及你申请这个项目是为了获得哪些能力

② 面向不同方向的包装重点

申请方向	包装重点	弱化内容
HCI硕士 (CMU/UW/UCL)	用户研究设计、可用性测试数据 无障碍的社会价值	后端工程细节
CS硕士 (UCSD/UMass)	全栈系统工程、算法复杂度分析 并发处理能力	设计美感
Spatial Computing (ASU/ETH Zurich)	数字孪生架构、空间感知算法 建筑背景的差异化优势	
智慧城市实习 (阿里云/商汤)	数据链路完整性、实地测绘经验 GIS+图论的组合	学术研究视角

③ 简历技能描述模板

可直接使用的简历条目（英文版）

- Designed and implemented a multi-modal accessible navigation system for Gulangyu UNESCO World Heritage Site, serving visually impaired users
- Built weighted graph routing engine (Dijkstra) with composite cost function incorporating slope, friction, and user-type parameters (normal/elderly/disabled/blind), validated via 1,000-case NLP stress test (accuracy: XX%) and 100-agent Unity A/B simulation
- Full-stack: Unity 3D (C#) + FastAPI (Python) + SQLite + React 19, achieving <500ms end-to-end response with concurrent agent load testing
- Reduced simulated hazard trigger rate by XX% (Group B vs Group A) through accessibility-first path planning algorithm

完整时间线总览

阶段	内容	预计时长	产出物	状态
Phase 0	测试前最终检查 验证所有API正常	30分钟	/api/health通过	
Phase 1	真实用户测试 5人×20分钟	1-2天	test_report.csv accuracy_chart.png	
Phase 2	缺陷修复 基于真实用户数据	1天	修复后重新通过测试	
Phase 3	1000条模拟压测 NLP鲁棒性验证	2天	confusion_matrix.png stress_results.csv	
Phase 4	Unity A/B仿真 100个Agent对照	1周	Demo视频 heatmap对比图	
Phase 5	数据整合报告 技术报告写作	3天	技术报告PDF 作品集素材包	
Phase 6	作品集包装 文书与简历	持续	PS文书初稿 简历技能更新	

你不是会写代码的建筑生，你是一个用空间认知构建感知系统的交叉领域工程师。这份手册的每一个步骤，都是在把这个判断从自我定义变成有数据支撑的事实。